

Uber Premium Booking Status Prediction

Multi-Class Classification Analysis

MGT256 Group 4

2026-02-01

Contents

Abstract	2
Data Import and Preparation	2
Load Required Libraries	2
Import and Merge Data	2
Remove leading/trailing spaces from all character columns	3
Standardize missing values and negative values to NA	3
Handle target variable (Booking.Status)	3
Standardize Date format and extract features	5
Standardize Time format and extract Hour	6
Check NA counts in predictors	6
Remove rows with NA in Traffic.Level and Avg.VTAT	7
Convert Categorical Variables to Factors	7
Data Visualizations	7
Graph 1: Cancellation Rate by Hour	7
Graph 2: Avg.VTAT Distribution	8
Graph 3: Avg.VTAT by Booking Status	9
Graph 4: Booking Volume by Hour	10
Logistic Regression	11
Stratified 60/40 Train-Validation Split	11
Fit Logistic Regression Models on training	12
Predicted probability of “Not Completed” on validation	12
Find Optimal Threshold (Log Transform Model)	13
Confusion Matrix at ROC-Chosen Threshold	14

Results and Discussion	18
Model Performance	18
Key Findings	18
Limitations	18
Improvement	18

Abstract

This analysis aims to predict Uber Premium Booking Status.

- Total samples: 20,795 rides
- Target variable: Booking Status with 2 categories (Completed and Non-Completed, including Cancelled by Driver, Cancelled by Customer, No Driver Found and Incomplete)
- Selected 7 core predictors: Booking Hour, Booking Month, Weekday, Vehicle Type, Traffic Condition and Booking Platform, Average Pickup Time.

Data Structure

- Source: Two datasets (booking_premium.csv, ride_premium.csv)
- Merge Key: Booking.ID
- Time Period: 2024-01-01 to 2024-12-30
- Features: 20+ variables including temporal, categorical, and numeric

Data Import and Preparation

Load Required Libraries

```
# Clear environment
rm(list = ls())

# Data manipulation
library(dplyr)
library(lubridate)
library(tidyr)
library(stringr)
library(caret)      # For createDataPartition

# Visualization
library(ggplot2)
library(pROC)       # For ROC curve and AUC
```

Import and Merge Data

```
# Import datasets
booking_data <- read.csv('booking_premium.csv')
ride_data <- read.csv('ride_premium.csv')

# Merge datasets by Booking ID
merged_data <- merge(booking_data,ride_data,by='Booking.ID')
```

Remove leading/trailing spaces from all character columns

```
# Check if column is character 'TRUE'
char_cols <- sapply(merged_data, is.character)

# Clean only columns where 'char_cols = TRUE' (remove extra spaces)
merged_data[char_cols] <- lapply(merged_data[char_cols], function(x) {
  x <- str_trim(x)          # Remove leading/trailing spaces
  x <- str_squish(x)        # Replace multiple spaces with single space
  return(x)
})
```

Standardize missing values and negative values to NA

```
# Define all representations of missing values
missing_values <- c("", " ", "null", "NA", "N/A", "na", "--", "nan", "NaN")

# Replace all missing value representations with NA
merged_data[] <- lapply(merged_data, function(column) {
  column[column %in% missing_values] <- NA
  return(column)
})

# Replace all negative values in Avg.VTAT with NA
merged_data <- merged_data %>%
  mutate(
    Avg.VTAT = ifelse(as.numeric(Avg.VTAT) < 0, NA, as.numeric(Avg.VTAT))
  )
```

Handle target variable (Booking.Status)

```
# Check Booking.Status
sum(is.na(merged_data$Booking.Status))
```

```
## [1] 0
```

```
table(merged_data$Booking.Status)
```

```
##
## Cancelled by Customer    Cancelled by Driver    Completed
##           1459           3642           13026
##           Incomplete     No Driver Found
##           1212           1456

# Create binary target:
merged_data$Target <- ifelse(merged_data$Booking.Status == "Completed",
                             "Completed", "Not Completed")

# Ensure Class is a factor with "Good" first and "Bad" second.
# -----
# In logistic regression (glm with family = binomial),
# predicted probabilities (type = "response") correspond to
# the *second* level of the outcome factor.
#
# By setting the levels as c("Good", "Bad"):
# - The model will output predicted probabilities for the "Bad" class,
#   which is the event of interest (the important/risk class).
# - These probabilities can be directly interpreted as P(Bad | predictors).
# - Sensitivity and specificity can then be computed with respect to
#   correctly identifying "Bad" credit applicants.
#
# If the order were reversed, the model would instead return P(Good),
# which would flip the interpretation of all predicted probabilities.

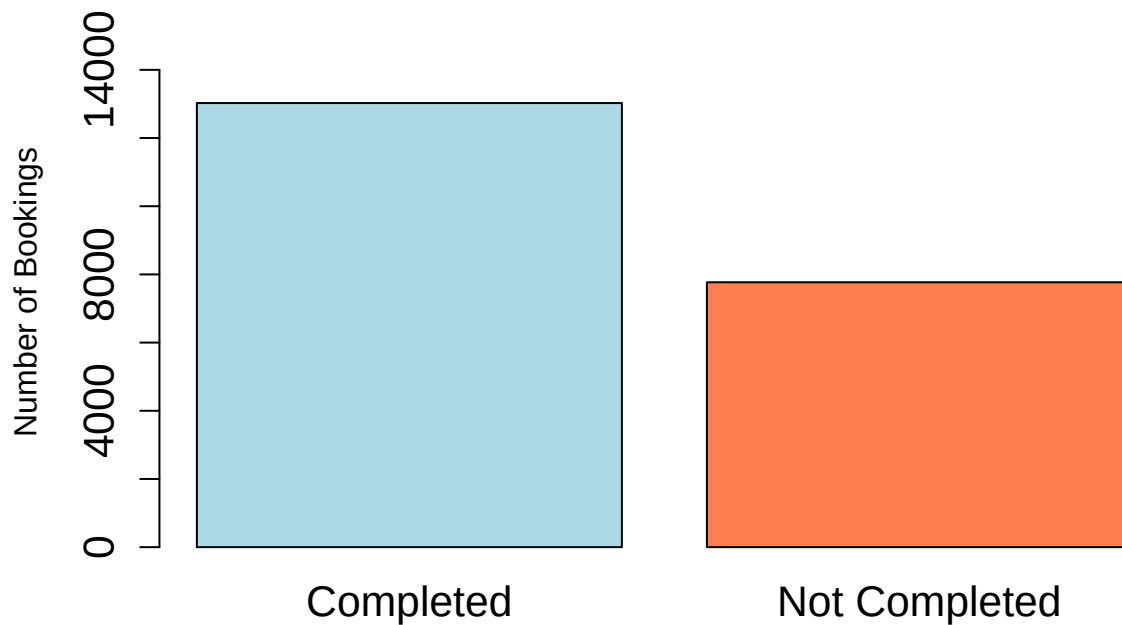
# Convert to factor (Completed first, Not Completed second)
merged_data$Target <- factor(merged_data$Target,
                             levels = c("Completed", "Not Completed"))
levels(merged_data$Target)
```

```
## [1] "Completed"    "Not Completed"
```

```
# Target distribution
target_counts <- table(merged_data$Target)
target_prop <- prop.table(target_counts) * 100

# Plot distribution
barplot(target_counts,
        main = "Booking Status Distribution",
        col = c("lightblue", "coral"),
        ylab = "Number of Bookings",
        cex.main = 1.8,
        cex.axis = 1.3,
        cex.names = 1.3,
        ylim = c(0, max(target_counts) * 1.15))
```

Booking Status Distribution



Standardize Date format and extract features

```
# Function to parse 3 date formats found in the data

parse_date <- function(date_string) {
  formats <- c(
    "%d-%b-%Y",      # 03-Mar-2024
    "%Y-%m-%d",      # 2024-01-15
    "%m/%d/%Y"       # 12/15/2024
  )

  result <- NA      # in failed cases
  for(fmt in formats) {
    parsed <- suppressWarnings(as.Date(date_string, format = fmt))
    if(!is.na(parsed)) {
      result <- parsed
      break # Stop the loop once we find a format that works
    }
  }
  return(result)
}

# Apply the date parsing function
merged_data$Date_Parsed <- sapply(merged_data$Date, parse_date) %>%
```

```

as.Date(origin = "1970-01-01")

# Extract temporal features
merged_data <- merged_data %>%
  mutate(
    Weekday = wday(Date_Parsed, label = TRUE),
    Month = month(Date_Parsed)
  )

```

Standardize Time format and extract Hour

```

# Function to parse 3 time formats found in the data

parse_time <- function(time_string) {
  formats <- c(
    "%H:%M:%S",      # 13:15:17
    "%H:%M",         # 21:15
    "%I:%M %p"       # 2:34 AM, 10:22 PM
  )
  result <- NA      # in failed cases
  for(fmt in formats) {
    parsed <- suppressWarnings(strptime(time_string, format = fmt))
    if(!is.na(parsed)) {
      result <- (as.numeric(format(parsed, "%H"))) # Extract hour (0-23)
      break
    }
  }
  return(result)
}

# Apply the function to extract hour from Time column
merged_data$Hour <- sapply(merged_data$Time, parse_time)

```

Check NA counts in predictors

```
sum(is.na(merged_data$Hour))
```

```
## [1] 0
```

```
sum(is.na(merged_data$Weekday))
```

```
## [1] 0
```

```
sum(is.na(merged_data$Month))
```

```
## [1] 0
```

```
sum(is.na(merged_data$Vehicle.Type))
```

```
## [1] 0
```

```
sum(is.na(merged_data$Booking.Platform))
```

```
## [1] 0
```

```
sum(is.na(merged_data$Traffic.Level))
```

```
## [1] 872
```

```
sum(is.na(merged_data$Avg.VTAT))
```

```
## [1] 1611
```

Remove rows with NA in Traffic.Level and Avg.VTAT

```
merged_data <- merged_data[!is.na(merged_data$Traffic.Level), ]  
merged_data <- merged_data[!is.na(merged_data$Avg.VTAT), ]
```

Convert Categorical Variables to Factors

```
# List of categorical predictors  
categorical <- c("Vehicle.Type", "Traffic.Level", "Booking.Platform", "Weekday")  
  
# Convert to factors  
merged_data[categorical] <- sapply(merged_data[categorical], as.factor)
```

Data Visualizations

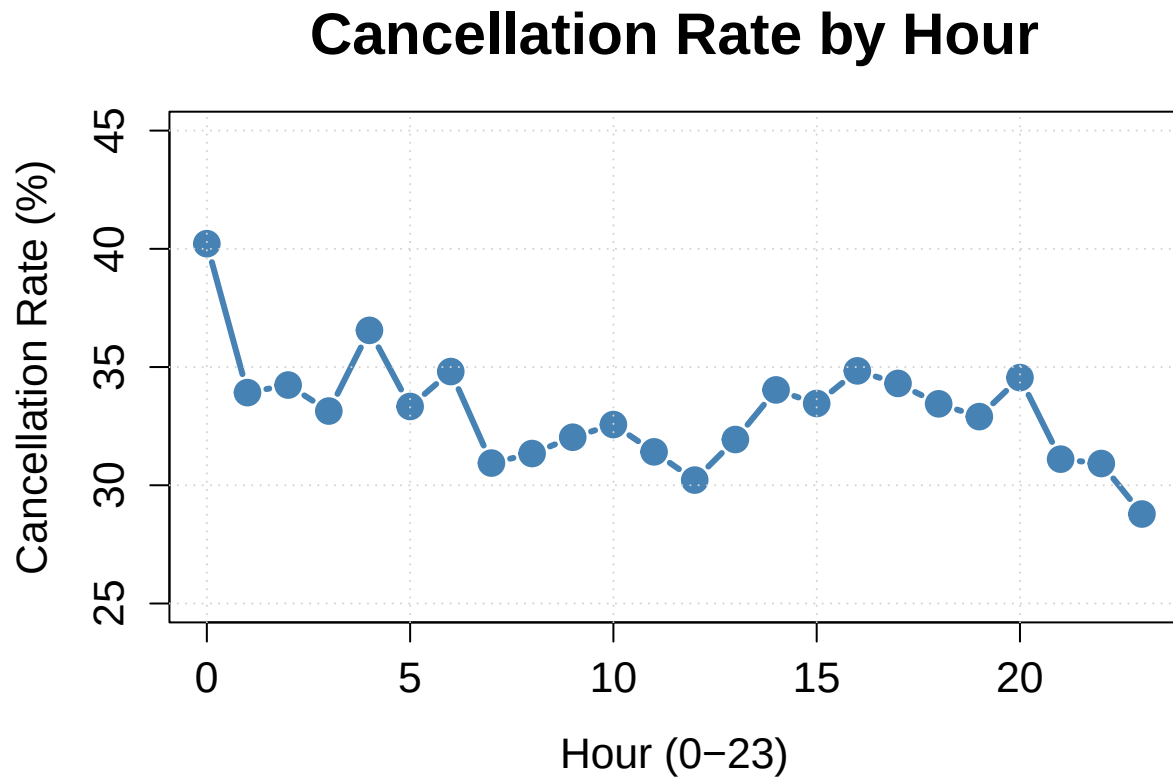
Graph 1: Cancellation Rate by Hour

```
hourly <- merged_data %>%  
  group_by(Hour) %>%  
  summarise(Cancel_Rate = mean(Target == "Not Completed") * 100)  
  
plot(hourly$Hour, hourly$Cancel_Rate,  
     type = "b",  
     col = "steelblue",  
     lwd = 3,  
     pch = 19,  
     cex = 1.5,
```

```

main = "Cancellation Rate by Hour",
xlab = "Hour (0-23)",
ylab = "Cancellation Rate (%)",
cex.main = 1.8,
cex.axis = 1.3,
cex.lab = 1.3,
ylim = c(25, 45))
grid()

```



Graph 2: Avg.VTAT Distribution

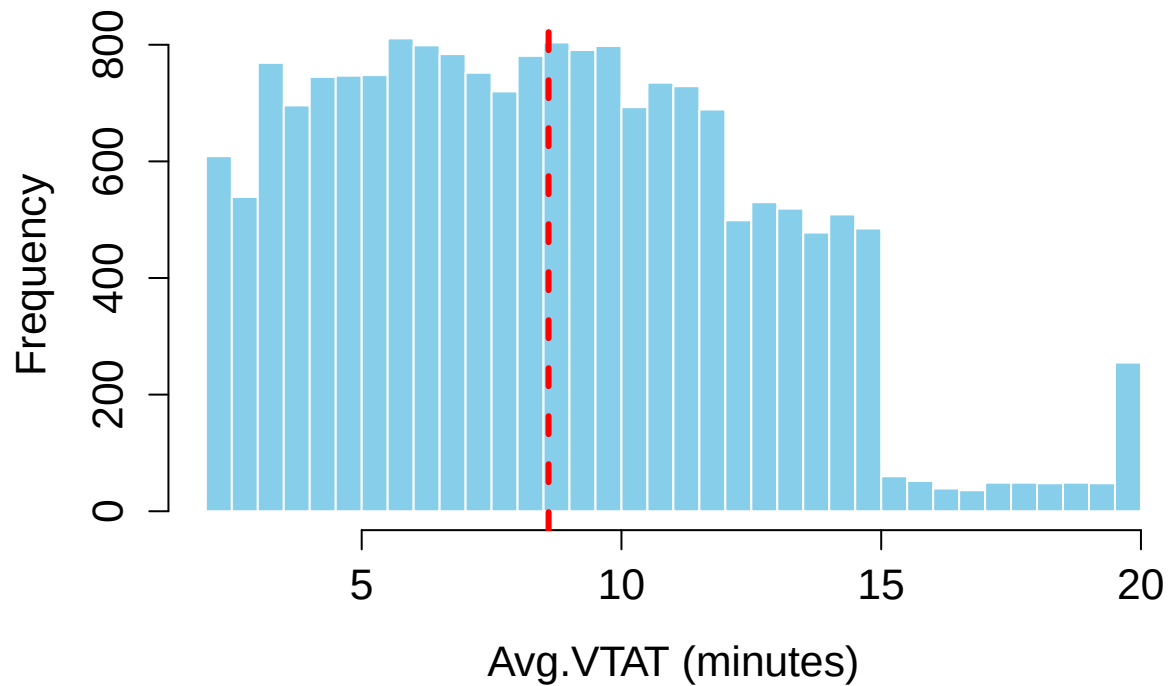
```

hist(merged_data$Avg.VTAT,
     breaks = 30,
     col = "skyblue",
     border = "white",
     main = "Distribution of Average Pickup Time",
     xlab = "Avg.VTAT (minutes)",
     ylab = "Frequency",
     cex.main = 1.8,
     cex.axis = 1.3,
     cex.lab = 1.3)

abline(v = mean(merged_data$Avg.VTAT), col = "red", lwd = 3, lty = 2)

```

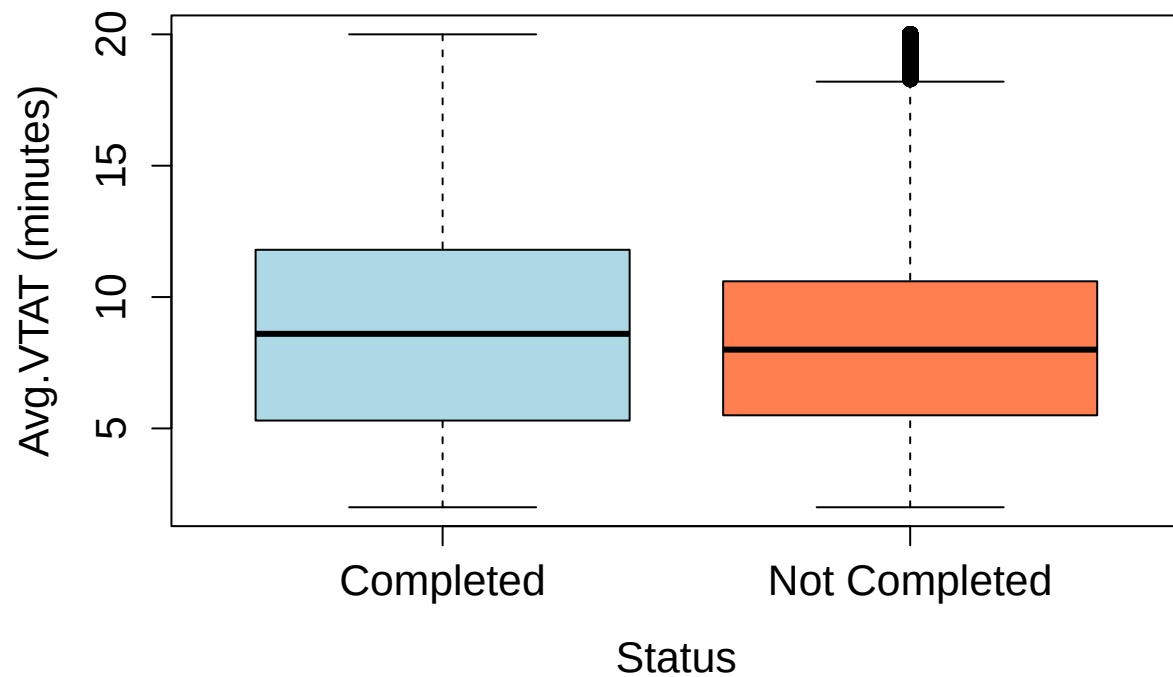

Distribution of Average Pickup Time



Graph 3: Avg.VTAT by Booking Status

```
boxplot(Avg.VTAT ~ Target,  
  data = merged_data,  
  col = c("lightblue", "coral"),  
  main = "Avg Pickup Time by Booking Status",  
  xlab = "Status",  
  ylab = "Avg.VTAT (minutes)",  
  cex.main = 1.8,  
  cex.axis = 1.3,  
  cex.lab = 1.3)
```

Avg Pickup Time by Booking Status

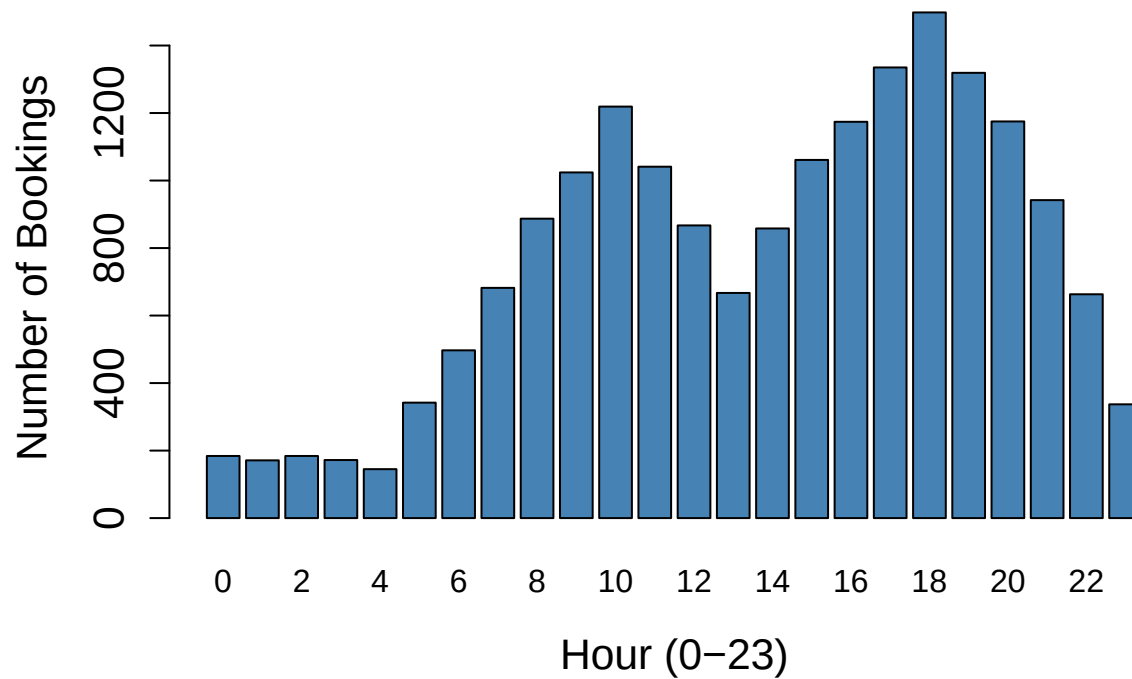


Graph 4: Booking Volume by Hour

```
hour_count <- table(merged_data$Hour)

barplot(hour_count,
        col = "steelblue",
        main = "Booking Volume by Hour",
        xlab = "Hour (0-23)",
        ylab = "Number of Bookings",
        cex.main = 1.8,
        cex.axis = 1.3,
        cex.lab = 1.3)
```

Booking Volume by Hour



Logistic Regression

Stratified 60/40 Train-Validation Split

```
set.seed(19)

idx <- createDataPartition(merged_data$Target, p = 0.6, list = FALSE)
train_data <- merged_data[idx, ]
valid_data <- merged_data[-idx, ]

print(prop.table(table(train_data$Target))*100)
```

```
##
##      Completed Not Completed
##      67.1546      32.8454
```

```
print(prop.table(table(valid_data$Target))*100)
```

```
##
##      Completed Not Completed
##      67.15467      32.84533
```

Fit Logistic Regression Models on training

```
# Model 1: Original
model1 <- glm(Target ~ Hour + Weekday + Month + Vehicle.Type +
              Traffic.Level + Booking.Platform + Avg.VTAT,
              data = train_data, family = binomial)

# Model 2: Add polynomial terms
model2 <- glm(Target ~ Hour + I(Hour^2) + Weekday + Month +
              Vehicle.Type + Traffic.Level + Booking.Platform +
              Avg.VTAT + I(Avg.VTAT^2),
              data = train_data, family = binomial)

# Model 3: Add interaction
model3 <- glm(Target ~ Hour * Avg.VTAT + Weekday + Month +
              Vehicle.Type + Traffic.Level + Booking.Platform,
              data = train_data, family = binomial)

# Model 4: Log transform
model4 <- glm(Target ~ Hour + Weekday + Month +
              Vehicle.Type + Traffic.Level + Booking.Platform +
              log(Avg.VTAT),
              data = train_data, family = binomial)
```

Predicted probability of “Not Completed” on validation

```
# Predictions
valid_prob1 <- predict(model1, newdata = valid_data, type = "response")
valid_prob2 <- predict(model2, newdata = valid_data, type = "response")
valid_prob3 <- predict(model3, newdata = valid_data, type = "response")
valid_prob4 <- predict(model4, newdata = valid_data, type = "response")

head(valid_prob1, 10)

##          11          32          34          43          44          51          53          56
## 0.3250220 0.2995332 0.3449294 0.3318288 0.3332956 0.3411098 0.3281816 0.3355511
##          60          61
## 0.3246078 0.3160163
```

```
head(valid_prob2, 10)

##          11          32          34          43          44          51          53          56
## 0.3708876 0.2966486 0.3444477 0.3338781 0.3372642 0.3205677 0.3490014 0.3385284
##          60          61
## 0.3272360 0.3120785
```

```
head(valid_prob3, 10)

##          11          32          34          43          44          51          53          56
```

```
## 0.3288189 0.2982413 0.3465282 0.3346585 0.3309572 0.3399293 0.3288295 0.3344063
##          60          61
## 0.3285318 0.3160472
```

```
head(valid_prob4, 10)
```

```
##          11          32          34          43          44          51          53          56
## 0.3395094 0.2960848 0.3395187 0.3233583 0.3267682 0.3411926 0.3098253 0.3317806
##          60          61
## 0.3336410 0.3188961
```

```
# Calculate AUC
auc1 <- auc(roc(valid_data$Target, valid_prob1))
auc2 <- auc(roc(valid_data$Target, valid_prob2))
auc3 <- auc(roc(valid_data$Target, valid_prob3))
auc4 <- auc(roc(valid_data$Target, valid_prob4))

# Compare
comparison <- data.frame(
  Model = c("Original", "Polynomial", "Interaction", "Log Transform"),
  AUC = c(auc1, auc2, auc3, auc4)
)

print(comparison)
```

```
##          Model          AUC
## 1      Original 0.5048626
## 2    Polynomial 0.5031196
## 3    Interaction 0.5052157
## 4 Log Transform 0.5080757
```

Find Optimal Threshold (Log Transform Model)

```
# Compute the ROC curve
roc_obj <- roc(response = valid_data$Target, predictor = valid_prob4)

# Optimal threshold
optimal_thres <- coords(roc_obj, x = "best",
                        best.method = "closest.topleft",
                        ret = "threshold")
optimal_thres

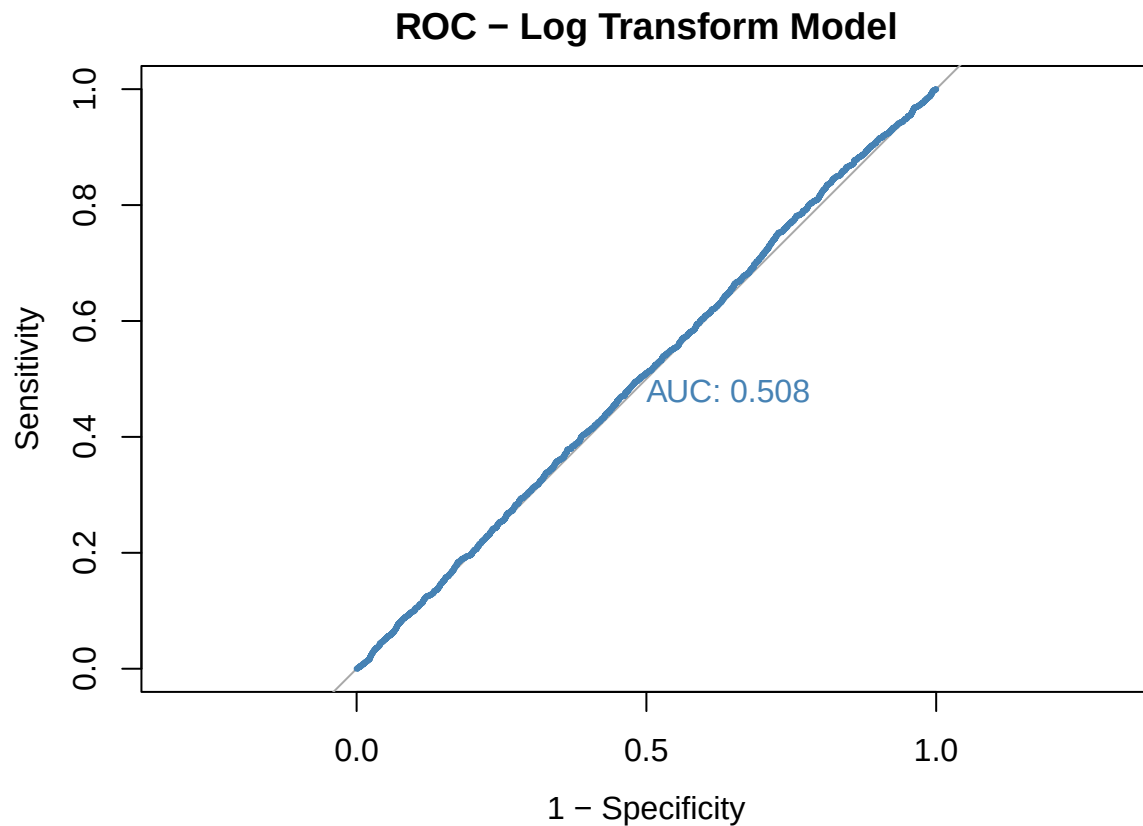
## threshold
## 1 0.3279808
```

```
# Plot the ROC curve with the AUC displayed
plot.roc(roc_obj,
         legacy.axes = TRUE,
         print.auc = TRUE,
```

```

main = "ROC - Log Transform Model",
lwd = 3,
col = 'steelblue')

```



Confusion Matrix at ROC-Chosen Threshold

```

# Classify using optimal threshold
pred_roc <- factor(ifelse(valid_prob4 > as.numeric(optimal_thres),
                          "Not Completed", "Completed"),
                  levels = c("Completed", "Not Completed"))

# Confusion matrix
cm_roc <- confusionMatrix(pred_roc,
                          valid_data$Target,
                          positive = "Not Completed")

print(cm_roc)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    Completed Not Completed
## Completed      2380      1200
## Not Completed  2574      1223

```

```
##
##          Accuracy : 0.4884
##          95% CI : (0.4769, 0.4999)
##    No Information Rate : 0.6715
##    P-Value [Acc > NIR] : 1
##
##          Kappa : -0.013
##
##    McNemar's Test P-Value : <2e-16
##
##          Sensitivity : 0.5047
##          Specificity : 0.4804
##    Pos Pred Value : 0.3221
##    Neg Pred Value : 0.6648
##          Prevalence : 0.3285
##    Detection Rate : 0.1658
##    Detection Prevalence : 0.5147
##    Balanced Accuracy : 0.4926
##
##    'Positive' Class : Not Completed
##
```

```
# =====
# KNN with Manual 10-Fold Cross-Validation
# =====

library(class) # for knn()
library(caret) # for createFolds() and confusionMatrix()

# 1) Build KNN datasets from existing train_data / valid_data
knn_train <- train_data %>%
  dplyr::select(Target, Hour, Weekday, Month,
                Vehicle.Type, Traffic.Level,
                Booking.Platform, Avg.VTAT) %>%
  na.omit()

knn_valid <- valid_data %>%
  dplyr::select(Target, Hour, Weekday, Month,
                Vehicle.Type, Traffic.Level,
                Booking.Platform, Avg.VTAT) %>%
  na.omit()

# Make sure Target is a factor with same levels
knn_train$Target <- factor(knn_train$Target)
knn_valid$Target <- factor(knn_valid$Target,
                           levels = levels(knn_train$Target))

# 2) Turn predictors into numeric matrix (dummy variables for factors)
x_train <- model.matrix(
  Target ~ Hour + Weekday + Month + Vehicle.Type +
  Traffic.Level + Booking.Platform + Avg.VTAT,
  data = knn_train
)[, -1] # drop intercept
```

```

x_valid <- model.matrix(
  Target ~ Hour + Weekday + Month + Vehicle.Type +
    Traffic.Level + Booking.Platform + Avg.VTAT,
  data = knn_valid
)[, -1]

y_train <- knn_train$Target
y_valid <- knn_valid$Target

# 3) Scale predictors (very important for KNN)
x_train_scaled <- scale(x_train)
x_valid_scaled <- scale(
  x_valid,
  center = attr(x_train_scaled, "scaled:center"),
  scale = attr(x_train_scaled, "scaled:scale")
)

# 4) 10-fold cross-validation to choose k
set.seed(19)
folds <- createFolds(y_train, k = 10, list = TRUE, returnTrain = FALSE)

k_values <- seq(3, 25, 2) # try odd k's: 3,5,7,...,25
cv_results <- data.frame(k = k_values, Accuracy = NA_real_)

for (i in seq_along(k_values)) {
  k <- k_values[i]
  acc_vec <- numeric(length(folds))

  for (f in seq_along(folds)) {
    val_idx <- folds[[f]]

    x_tr <- x_train_scaled[-val_idx, , drop = FALSE]
    y_tr <- y_train[-val_idx]
    x_te <- x_train_scaled[val_idx, , drop = FALSE]
    y_te <- y_train[val_idx]

    pred <- knn(
      train = x_tr,
      test = x_te,
      cl = y_tr,
      k = k
    )

    acc_vec[f] <- mean(pred == y_te)
  }

  cv_results$Accuracy[i] <- mean(acc_vec)
}

cv_results

```

```

##      k Accuracy
## 1    3 0.6063091

```



```
## 2    5 0.6135376
## 3    7 0.6230245
## 4    9 0.6281750
## 5   11 0.6328737
## 6   13 0.6411839
## 7   15 0.6449809
## 8   17 0.6481434
## 9   19 0.6476925
## 10  21 0.6497708
## 11  23 0.6547389
## 12  25 0.6550990
```

```
# 5) Pick the best k
best_k <- cv_results$k[which.max(cv_results$Accuracy)]
best_k
```

```
## [1] 25
```

```
# 6) Train final KNN on full training data and evaluate on validation set
knn_valid_pred <- knn(
  train = x_train_scaled,
  test  = x_valid_scaled,
  cl     = y_train,
  k      = best_k
)
```

```
# Confusion matrix on validation set
knn_cm <- confusionMatrix(
  knn_valid_pred,
  y_valid,
  positive = "Not Completed"
)
knn_cm
```

```
## Confusion Matrix and Statistics
```

```
##
##               Reference
## Prediction    Completed Not Completed
## Completed      4735      2285
## Not Completed   219       138
##
##               Accuracy : 0.6606
##               95% CI : (0.6496, 0.6714)
##               No Information Rate : 0.6715
##               P-Value [Acc > NIR] : 0.9781
##
##               Kappa : 0.0163
##
## Mcnemar's Test P-Value : <2e-16
##
##               Sensitivity : 0.05695
##               Specificity : 0.95579
##               Pos Pred Value : 0.38655
```

```
##          Neg Pred Value : 0.67450
##          Prevalence : 0.32845
##          Detection Rate : 0.01871
##    Detection Prevalence : 0.04839
##          Balanced Accuracy : 0.50637
##
##          'Positive' Class : Not Completed
##
```

Results and Discussion

Model Performance

Using the log-transformed model with ROC-optimal threshold:

- **AUC:** 0.5080757 (marginally above random guessing at 0.50)
- **Accuracy:** 0.4884
- **Sensitivity:** 0.5047 (detects ~50% of cancellations)
- **Specificity:** 0.4804
- **Precision:** 32.2% (only ~32% of predictions are correct)

Key Findings

1. **Weak Predictive Power:** All models achieved AUC ~ 0.50-0.51, indicating performance barely better than random guessing.
2. **Model Transformations:** Polynomial terms, interactions, and log transformation provided negligible improvements (+0.003 AUC), confirming the fundamental limitation is weak features, not model specification.
3. **Class Imbalance:** The 63%/37% split required threshold optimization, but even with optimal thresholds, the model remains weak.

Limitations

- Current features lack strong predictive signals
- Missing key historical data: customer booking history, driver-side predictors, trip distance
- Logistic regression may not capture complex patterns

Improvement

1. **Feature Engineering:**
 - Add customer-level features (past cancellation rate, booking frequency)
 - Incorporate driver-side variables (acceptance rate, distance to pickup)
2. **Advanced Methods:**
 - Explore advanced models to capture relationships
3. **Data Collection:**
 - Enhance dataset with richer behavioral and contextual features
 - Track customer engagement metrics and historical patterns